



Panadería POS

Dashboard de ventas y punto de venta con
microservicios REST



Redes e Infraestructura · Proyecto de curso – 1ª parte
Victoria Herrera · Jhon Sons
Octubre 2026



Cómo se desarrolló el proyecto

Metodología sencilla en cuatro fases, como pide el enunciado



20%

Fase 1

Definición

Problema, dataset,
usuarios y
requerimientos



20%

Fase 2

Diseño

Arquitectura de
microservicios, datos y
componentes



30%

Fase 3

Implementación y pruebas

3 microservicios + front-
end, 105 pruebas



30%

Fase 4

Despliegue

2 servidores + DNS,
acceso por dominio

FASE 1

Definición

Problema, dataset, usuarios y requerimientos



El problema

FASE 1

Una panadería de barrio registra sus ventas, pero no las aprovecha

“El dueño no sabe qué productos le dejan más dinero, qué días vende más, quiénes son sus clientes frecuentes ni qué productos no se venden. El cajero registra ventas sin ninguna validación.”

11 necesidades identificadas → 16 requerimientos funcionales



Ver de un vistazo cuánto vendió

Ingresos, ventas, unidades, ticket promedio



Saber qué hornear

Productos más vendidos y productos sin ventas



Conocer a sus clientes

Ranking de clientes por compras y gasto



Controlar el catálogo

Precios, tipo de venta e historial de cambios



Vender rápido y sin errores

Carrito, clientes y validación de reglas



El dataset

FASE 1

Kaggle · “Venta de productos de panadería” (cuantico)

42

productos

con precio por unidad o
por kg

210

clientes

solo identificados por
número

1.203

ventas

del lunes 2 al sábado 7 de
marzo de 2020

€ 1.229

en ingresos

13 productos nunca se
vendieron

Adaptaciones al cargarlo (los CSV no se modifican)

- 7 productos se venden por peso (€/kg, el cajero ingresa gramos); “Huevos” pasa a “Huevos (docena)”
- Los clientes del dataset se llaman “Cliente N”; se agrega “Consumidor final”
- Cada venta guarda el precio del momento: si el precio cambia, el historial no se altera



Dos tipos de usuario

FASE 1

Cada uno ve solo lo que le corresponde



Administrador

Dueño de la panadería

- Dashboard con indicadores y gráficas
- Filtro por rango de fechas
- Catálogo: crear, editar, desactivar
- Historial de cambios del catálogo
- Gestión de usuarios del sistema



Cajero

Vendedor de mostrador

- Venta con carrito (unidades o gramos)
- Buscar cliente por número o nombre
- Registrar cliente nuevo solo con el nombre
- Venta a “Consumidor final”
- Sus ventas y total del día



Requerimientos y reglas de negocio

FASE 1

Todo lo que el sistema debe cumplir — y que después se prueba

16 requerimientos
funcionales

8 requerimientos no
funcionales

21 reglas de negocio

Algunas reglas clave

RN01 Solo se vende un producto que exista y esté activo

RN04 Por peso: $\text{precio_kg} \times \text{gramos} \div 1000$

RN05 El total lo calcula el back-end, nunca el navegador

RN11 Los productos no se borran: se desactivan

RN12 Todo cambio del catálogo queda en el historial

RN15 La venta es todo o nada

RN16 Total = suma exacta de los subtotales

FASE 2

Diseño

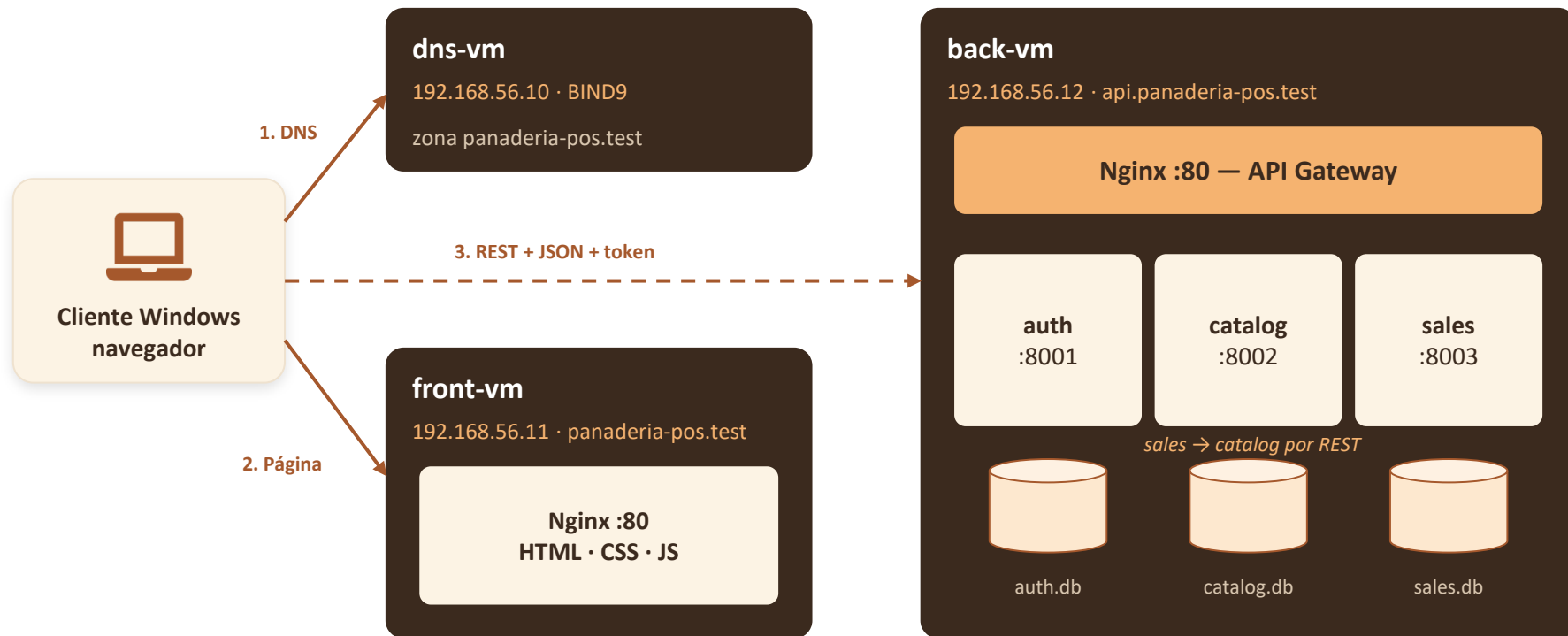
Arquitectura de microservicios, componentes y modelo de datos



Arquitectura del sistema

FASE 2

Microservicios accedidos por API REST, en tres máquinas virtuales





Decisiones de diseño

FASE 2

Por qué está construido así



Una base por microservicio

Ningún servicio lee la base de otro; se piden los datos por REST



Token JWT, API sin estado

El servidor no guarda sesiones; cada servicio verifica el token y el rol



Nginx como API Gateway

El front conoce una sola dirección; los puertos internos no se exponen



Dominio .test con DNS propio

.test es reservado para pruebas; .local no sirve en Mac (mDNS)



Venta como ticket

Encabezado + líneas con copia del precio: carrito y RN06



Vagrant + Ansible local

La misma caja funciona en Windows (Intel) y en Mac M1



Microservicios y endpoints REST

FASE 2

Sustantivos en las rutas, el método define la acción · 21 endpoints

auth-service

`/api/v1/auth`

`POST /tokens (login)`

`GET /users/me`

`GET /users`

`POST /users`

`PATCH /users/{id}`

catalog-service

`/api/v1/catalog`

`GET /products`

`GET /products/{id}`

`POST /products`

`PUT /products/{id}`

`PATCH /products/{id}`

`GET /changes`

sales-service

`/api/v1/sales`

`GET · POST /customers`

`GET /customers/{id}`

`POST /orders (carrito)`

`GET /orders · /orders/{id}`

`GET /metrics/summary`

`GET /metrics/... (4 más)`

Sin DELETE de productos ni usuarios (se desactivan con PATCH) · el historial solo tiene GET · códigos 200 · 201 · 400 · 401 · 403 · 404 · 409 · 422 · 503



Arquitectura en 4 capas

FASE 2

La misma estructura del proyecto de referencia de clase (mini_tasks_rest)

Presentación

api/

Rutas HTTP, token y rol, errores → códigos

Aplicación

application/

DTOs de entrada y salida (Pydantic)

Dominio

domain/

Modelos, contratos (ABC) y reglas de negocio

Infraestructura

infrastructure/

SQLite + SQLAlchemy, carga del dataset, cliente HTTP

Inversión de dependencias

OrderService (dominio) necesita los productos, pero solo conoce el contrato

ProductCatalog.

HttpProductCatalog lo implementa pidiendo los datos a catalog-service por REST.

En las pruebas se reemplaza por un catálogo falso: igual que TaskRepository / SQLAlchemyTaskRepository en clase.



Modelo de datos

FASE 2

Cada microservicio es dueño de sus datos · sin llaves foráneas entre bases



auth.db

usuarios

id · nombre · usuario (único) ·
password_hash · rol · activo



catalog.db

productos

id · nombre (único) · precio · tipo_venta ·
activo

cambios_producto

producto · acción · campo · antes · después
· usuario · fecha



sales.db

clientes

id (0 = Consumidor final) · nombre

ventas

id · cliente · cajero · fecha · total

lineas_venta

venta · producto · nombre · tipo · cantidad ·
precio · subtotal

FASE 3

Implementación y pruebas

3 microservicios en FastAPI, front-end en HTML/CSS/JS y 105 pruebas

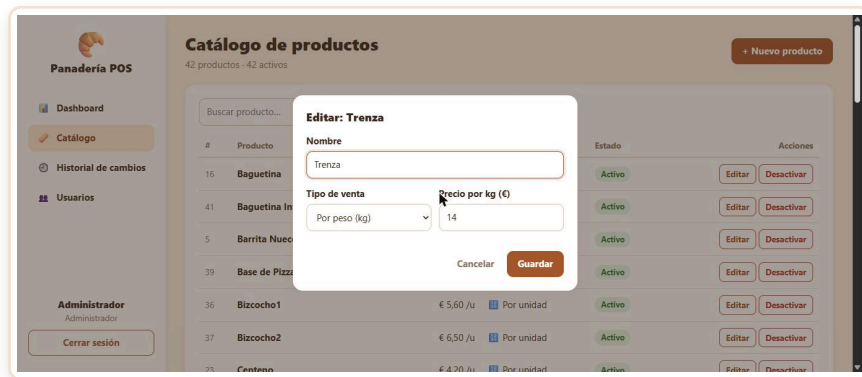
Front-end: administrador

FASE 3

Dashboard con métricas del dataset · catálogo con historial de cambios



Indicadores, ventas por día, top productos, top clientes y productos sin ventas



Editar precio y tipo de venta; cada cambio queda registrado con usuario y fecha

- Login con rol: cada usuario entra a su pantalla
- El dashboard reproduce los números del dataset: 1.203 ventas, € 1.229,40, 203 clientes



Front-end: cajero

FASE 3

Venta con carrito, por unidades o por gramos

 **Panadería POS**

Cajero: Cajero Cerrar sesión

Registrar venta

CLIENTE

Eddy Baez #211 Cambiar ×

AGREGAR PRODUCTO

Producto

Trenza — € 14,00 /kg

Gramos

500

Agregar

Se vende por peso: € 14,00 por kg. Ingrese los gramos.

CARRITO

Producto	Cantidad	Precio	Subtotal	
Bagueta	2 u	€ 0,60 /u	€ 1,20	×
Croissant	1 u	€ 1,00 /u	€ 1,00	×
Trenza	500 g	€ 14,00 /kg	€ 7,00	×

Ventas de hoy Actualizar

Ventas

1

Total del día

€ 31,20

Hora	Cliente	Productos	Total
10:05 a. m.	Consumidor final #0	Colon Pequeño	€ 31,20



Cliente por número o nombre

Eddy Baez quedó registrado con el #211



Unidades o gramos

Trenza: $500 \text{ g} \times € 14,00/\text{kg} = € 7,00$



Un solo cobro

Una petición con todo el carrito; el back recalcula y valida



Todo o nada

Si una línea falla, no se guarda nada

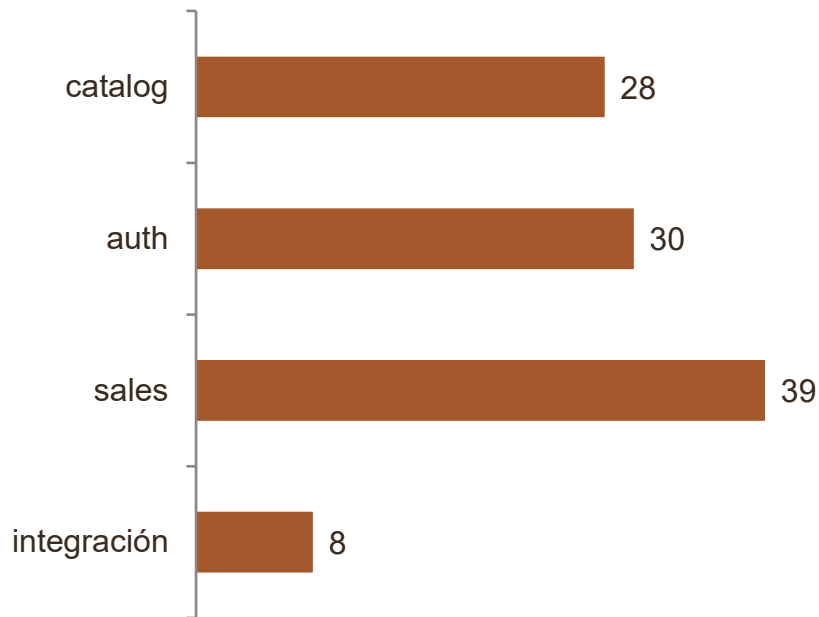


Pruebas: 105 de 105 aprobadas

FASE 3

Cada requerimiento y regla de negocio tiene al menos una prueba automática

Pruebas aprobadas por suite



Unitarias

Reglas del dominio con dobles en memoria: sin base de datos ni HTTP

De API

Cada servicio completo con el dataset real: códigos 201, 400, 401, 403, 404, 409, 422

Integración

Los 3 servicios juntos por HTTP; también se corrieron contra el servidor desplegado

Manuales

Flujos de administrador y cajero desde el navegador

FASE 4

Despliegue

Dos servidores (back y front) + DNS, acceso desde Windows por dominio



Despliegue con Vagrant y Ansible

FASE 4

Un solo comando crea y configura las tres máquinas: `vagrant up`



dns-vm

192.168.56.10

BIND9 · zona panaderia-pos.test · dns.yml



back-vm

192.168.56.12

3 servicios systemd + Nginx gateway · back.yml



front-vm

192.168.56.11

Nginx con el front-end · front.yml

Resultado del aprovisionamiento

```
dns-vm    : ok=9   changed=5   failed=0
back-vm   : ok=19  changed=15  failed=0
front-vm  : ok=10  changed=7   failed=0
```

- Caja bento/ubuntu-22.04: misma configuración en Windows y en Mac M1
- Ansible corre dentro de cada máquina (`ansible_local`)
- Servicios escuchan solo en 127.0.0.1: desde fuera, solo por el gateway
- Las bases viven fuera del código: redespregar no borra datos

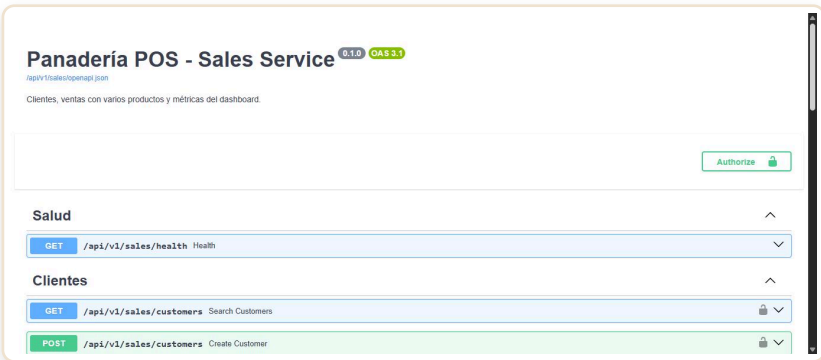


Acceso desde Windows por dominio

FASE 4

Pruebas reales desde un cliente Windows: PowerShell y navegador

```
PS> Resolve-DnsName panaderia-pos.test
panaderia-pos.test  A    192.168.56.11
PS> Resolve-DnsName api.panaderia-pos.test
api.panaderia-pos.test  A    192.168.56.12
PS> curl.exe http://api.panaderia-pos.test/
api/v1/sales/health
{"status":"ok","service":"sales"}
```



Swagger del back por el gateway: api.panaderia-pos.test/api/v1/sales/docs



Dos servidores: back-vm (back) y front-vm (front)



Pruebas de acceso desde un cliente Windows



Acceso por el dominio panaderia-pos.test



Resultado

3

microservicios

21

endpoints REST

105

pruebas aprobadas

3

máquinas virtuales

Aplicación: <http://panaderia-pos.test>

API: <http://api.panaderia-pos.test>

Código: github.com/oraboni08/proyecto-panaderia-pos

¡Gracias! · Demo en vivo

